Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación
Ingeniería en Sistemas de Información

Entregable 1: Diseño de infraestructura
Esquemático de nodos, servicios y segmentos de red

[Nombre del integrante 1] [Carné]
[Nombre del integrante 2] [Carné]
[Nombre del integrante 3] [Carné]

Arquitectura de Computadores II, sección 040
[Nombre del catedrático]
31 de octubre de 2026

Índice

Diseño de infraestructura 1
Propósito del documento 1
Modelo de despliegue 1
Distribución por nodos 2
Servicios 3
Esquemático de nodos, servicios y red 3
Segmentos de red 5
Comunicación entre componentes 6
Topics de Kafka 7
Almacenamiento 7
Perfiles de despliegue 8
Capacidad y recursos 8
Crecimiento a varios servidores 9
Referencias 10

Diseño de infraestructura

Propósito del documento

Este documento presenta el diseño de infraestructura del sistema de monitoreo perimetral urbano para la Municipalidad de San Juan Sacatepéquez: el esquemático de nodos, servicios y segmentos de red, la asignación de cada servicio a un nodo, los puertos y protocolos con que se comunican, el almacenamiento y la forma de crecer a varios servidores. Corresponde al primer entregable de la sección 8 del enunciado y se apoya en la decisión de plataforma registrada en el ADR-01 (Entregable 2).

Modelo de despliegue

El sistema se despliega on-premise en un solo servidor propio de la municipalidad. Todos los servicios corren en contenedores Docker (Merkel, 2014) y se comunican entre sí únicamente por red, por lo que la arquitectura es distribuida a nivel lógico aunque comparta máquina. Esta elección responde al cliente: un municipio mediano con presupuesto limitado, que maneja datos personales (placas y ubicaciones) y que no debe depender del internet para recibir y procesar alertas.

Tabla 1

Servidor físico del prototipo

Elemento	Valor
Equipo	Laptop Acer Predator (servidor del prototipo)
Procesador	Intel Core i7-13650HX, 20 hilos
Memoria	32 GB de RAM (20 GB asignados a Docker)
Sistema operativo	Windows 11 con Docker Desktop sobre WSL2 (núcleo Linux)
Recursos para contenedores	20 GB de RAM, 12 núcleos y 4 GB de intercambio (archivo .wslconfig)
Imágenes de contenedores	Linux (Debian y Alpine), con versiones fijas
Recomendado para producción	Servidor de 8 a 16 núcleos, 32 a 64 GB de RAM, discos en espejo, UPS y Ubuntu Server 22.04 LTS

Distribución por nodos

La sección 6 del enunciado propone una distribución de referencia en nodos. El proyecto la respeta como separación lógica y la despliega en un solo servidor físico (Tabla 2) y el esquemático de la Figura 1 la muestra gráficamente. Gracias a esa separación, si la municipalidad adquiere más equipos, cada nodo puede pasar a su propia máquina sin cambiar el código.

Tabla 2

Distribución de servicios por nodo

Nodo	Función primaria (enunciado)	Servicios del proyecto	SO recomendado
A	Kafka, Airflow, PostgreSQL, Redis	kafka, kafka-init, postgres, redis, airflow	Ubuntu Server 22.04
B	Microservicios, Elastic, Grafana	nginx, ingestor-1, ingestor-2, ingestor-3, correlador, despachador, aseguradora-mock, elasticsearch, grafana, kafka-ui	Ubuntu Server 22.04 (enunciado: macOS o Linux)
C/D	Generación de eventos (clientes)	simulador (scripts Python) y JMeter	Windows o Linux (equipos de los alumnos)

Nota. En el prototipo los tres nodos comparten el mismo servidor físico (Windows 11 + Docker Desktop con WSL2), decisión registrada en el ADR-01.

Servicios

Tabla 3

Servicios desplegados

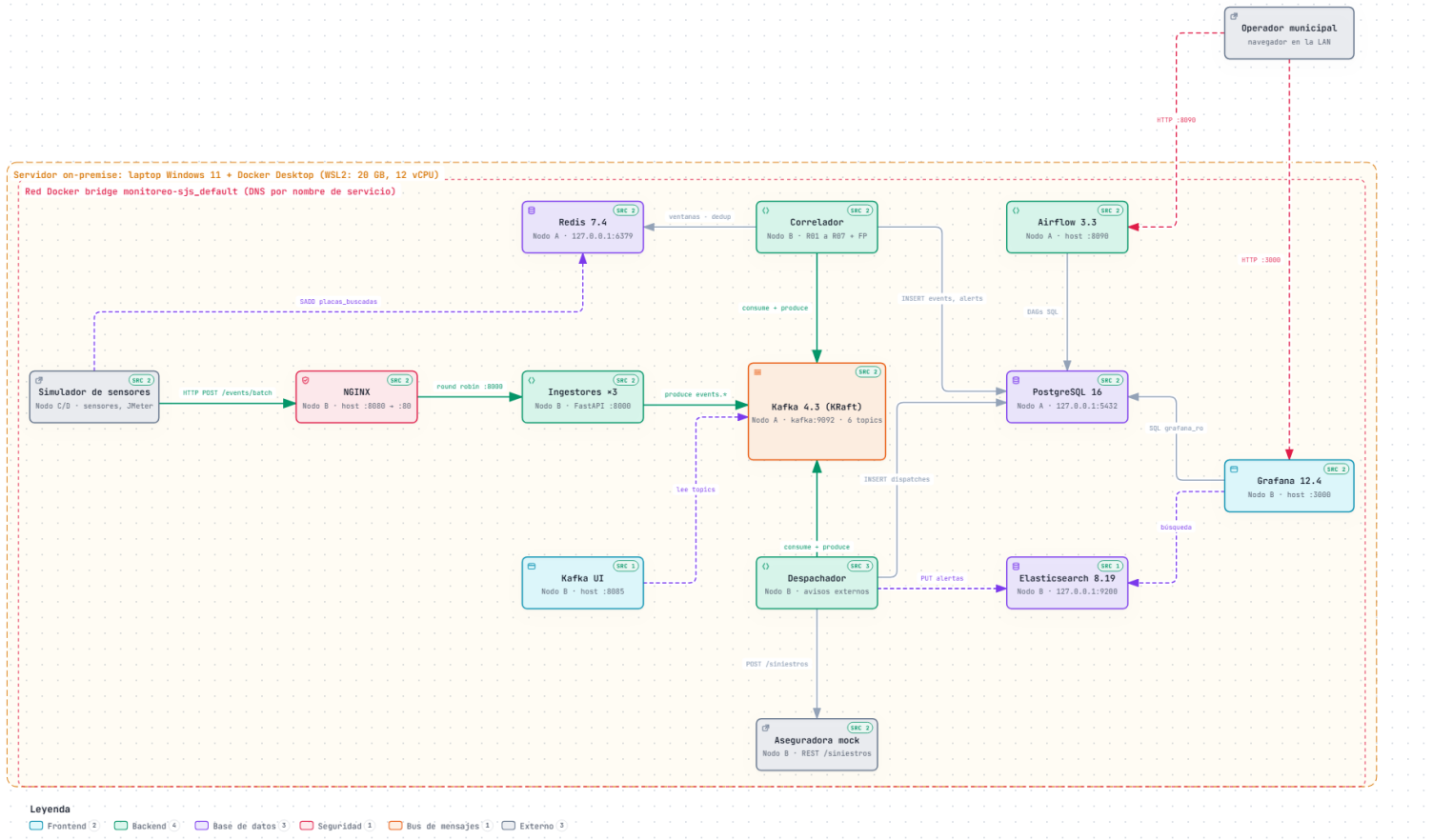
Servicio	Imagen y versión	Rol	Nodo
nginx	nginx:1.30.5-alpine	Balanceador: única puerta de entrada de eventos	B
ingestor-1/2/3	monitoreo-sjs/ingestor:1.0	API REST que valida, estandariza y publica en Kafka	B
kafka	apache/kafka:4.3.1	Broker de eventos en modo KRaft	A
kafka-init	apache/kafka:4.3.1	Crea los seis topics y termina	A
kafka-ui	kafbat/kafka-ui:v1.5.0	Interfaz web para ver topics y mensajes	B
correlador	monitoreo-sjs/correlador:1.0	Aplica las reglas de correlación	B
redis	redis:7.4.11	Estado en memoria del correlador	A
postgres	postgres:16.15	Base de datos principal	A
despachador / despachador-es	monitoreo-sjs/despachador:1.0	Avisa a las instituciones; la versión -es indexa en Elasticsearch	B
aseguradora-mock	monitoreo-sjs/aseguradora-mock:1.0	Entidad externa simulada (REST)	B
elasticsearch	elasticsearch:8.19.22	Búsqueda histórica de alertas	B
grafana	grafana/grafana:12.4.12	Visualización en tiempo real	B
airflow	apache/airflow:3.3.2-python3.12	Tareas periódicas	A
simulador	monitoreo-sjs/simulador:1.0	Generación de eventos a demanda	C/D

Esquemático de nodos, servicios y red

La Figura 1 es el esquemático de la infraestructura. Cada caja es un contenedor e indica a qué nodo lógico pertenece y por qué puerto se accede. Las dos fronteras punteadas representan el servidor físico y la red interna de Docker; el operador municipal queda fuera de ambas. Las flechas indican quién abre la conexión (cliente hacia servidor).

Figura 1

Esquemático de infraestructura: nodos, servicios y segmentos de red



Nota. Elaboración propia, generado a partir de la configuración del repositorio. La versión interactiva, con las referencias al código de cada componente, está en el archivo diagrama-infraestructura.html.

Segmentos de red

El diseño distingue tres segmentos de red con distinto nivel de exposición, de acuerdo con el principio de exponer solo lo necesario (Tabla 4).

Tabla 4

Segmentos de red

Segmento	Quién se conecta	Qué se expone
1. Red local de la municipalidad (LAN)	Operadores con navegador y sensores o simuladores externos	Solo los puertos de uso humano y de entrada de eventos: 8080, 3000, 8085 y 8090
2. Interfaz local del servidor (127.0.0.1)	Solo el propio servidor (administración y depuración)	Bases de datos y broker: 5432, 6379, 9094 y 9200
3. Red interna de Docker (bridge monitoreo-sjs_default)	Los contenedores entre sí, por nombre de servicio (DNS interno de Docker)	Todos los puertos internos; los ingestores, el correlador, el despachador y la aseguradora no publican ningún puerto fuera de esta red

Tabla 5

Puertos publicados en el servidor

Puerto en el servidor	Servicio (puerto interno)	Segmento	Uso
8080	nginx (80)	LAN	Entrada de eventos: POST /events y /events/batch
3000	grafana (3000)	LAN	Dashboards para el operador
8085	kafka-ui (8080)	LAN	Ver topics y mensajes
8090	airflow (8080)	LAN	Interfaz de tareas periódicas
5432	postgres (5432)	127.0.0.1	Administración de la base de datos
6379	redis (6379)	127.0.0.1	Depuración del estado del correlador
9094	kafka (9094)	127.0.0.1	Clientes de Kafka desde el servidor
9200	elasticsearch (9200)	127.0.0.1	Consultas directas al índice de alertas

Comunicación entre componentes

Tabla 6

Conexiones, protocolos y puertos internos

Origen	Destino	Protocolo y puerto	Propósito
Simulador / JMeter	nginx	HTTP :80 (8080 en el servidor)	Enviar lotes de eventos
nginx	ingestor-1/2/3	HTTP :8000	Reparto en turnos (round robin)
Ingestores	kafka	Kafka :9092	Publicar eventos válidos e inválidos
Correlador	kafka	Kafka :9092	Leer eventos y publicar alertas
Correlador	redis	RESP :6379	Ventanas, vehículos y duplicados
Correlador	postgres	PostgreSQL :5432	Guardar eventos, alertas y violaciones de SLO
Despachador	kafka	Kafka :9092	Leer alertas e inválidos; publicar avisos a instituciones
Despachador	aseguradora-mock	HTTP :8000	POST /siniestros
Despachador	elasticsearch	HTTP :9200	Indexar alertas
Despachador	postgres	PostgreSQL :5432	Registrar envíos e inválidos
Grafana	postgres / elasticsearch	PostgreSQL :5432 / HTTP :9200	Consultas de los dashboards
Airflow	postgres	PostgreSQL :5432	Tareas periódicas
kafka-ui	kafka	Kafka :9092	Mostrar topics y mensajes

Topics de Kafka

Tabla 7

Topics del broker

Topic	Particiones	Clave	Contenido
events.standardized	3	sector	Eventos válidos estandarizados
events.invalid	1	sector o sin-zona	Eventos rechazados con su motivo (dead-letter)
correlated.alerts	3	sector	Alertas generadas por el correlador
external.pnc	1	sector	Avisos a la PNC
external.bomberos	1	sector	Avisos a Bomberos
external.conred	1	sector	Avisos a CONRED

Nota. Retención de 7 días; la creación automática de topics está desactivada y los crea el contenedor kafka-init (Kreps et al., 2011).

Almacenamiento

Tabla 8

Volúmenes persistentes

Volumen	Servicio	Contenido
kafka-datos	kafka	Registro de mensajes de los topics (7 días)
postgres-datos	postgres	Eventos, alertas, envíos, estadísticas y auditoría
redis-datos	redis	Archivo AOF con el estado del correlador
elasticsearch-datos	elasticsearch	Índice de búsqueda de alertas
grafana-datos	grafana	Estado interno de Grafana
airflow-datos	airflow	Metadatos (SQLite) y registros de las tareas

Nota. Los volúmenes sobreviven a la recreación de los contenedores; solo se borran con docker compose down -v.

Perfiles de despliegue

Tabla 9

Perfiles de Docker Compose

Perfil	Servicios	Uso
core	kafka, kafka-init, kafka-ui, postgres, redis, ingestores, nginx, correlador, despachador, aseguradora-mock y grafana	Desarrollo y pruebas rápidas
full	Todo lo de core más elasticsearch y airflow (el despachador se reemplaza por despachador-es)	Pruebas finales y demostración
simulador	simulador (se ejecuta a demanda y termina)	Generación de eventos

Capacidad y recursos

Cada contenedor tiene un límite de memoria para que un servicio descontrolado no afecte a los demás. La suma de los límites del perfil completo es de unos 12 GB, dentro de los 20 GB asignados. Con la carga máxima sostenida medida (508 eventos por segundo), el sistema completo usó unos 4.6 GB de memoria y menos de un núcleo de los doce disponibles; el detalle de las pruebas está en el informe final (Entregable 7).

Tabla 10

Límites de memoria por servicio

Servicio	Límite	Uso medido bajo carga
airflow	3 GB	1.34 GB
elasticsearch	2 GB	1.6 GB
kafka	1.5 GB	0.45 GB
postgres	1 GB	0.24 GB
kafka-ui	768 MB	336 MB
correlador, redis, kafka-init, simulador, grafana	512 MB c/u	de 24 a 182 MB
ingestor-1/2/3	384 MB c/u	de 40 a 54 MB
despachador	256 MB	24 MB
nginx, aseguradora-mock	128 MB c/u	28 y 48 MB

Crecimiento a varios servidores

Si la municipalidad incorpora más equipos, el diseño permite repartir los nodos sin modificar el código:

1. Instalar Docker en cada servidor y copiar el repositorio.
2. Llevar los servicios del nodo A a un servidor, los del nodo B a otro y los generadores a los equipos de los clientes (nodos C y D).
3. Cambiar los nombres de servicio por las direcciones IP o nombres de red correspondientes en las variables de entorno (por ejemplo KAFKA_BOOTSTRAP, POSTGRES_HOST y REDIS_HOST) y anunciar el listener externo de Kafka con la dirección del servidor A.
4. Abrir en el firewall solo los puertos necesarios entre servidores (9092, 5432, 6379, 9200 y 8000) y mantener en la LAN únicamente 8080, 3000, 8085 y 8090.
5. Con tres servidores o más, convertir Kafka en un clúster de tres brokers con factor de replicación 3 para eliminar el punto único de falla del broker.

Referencias

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. En *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*.

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal, 2014*(239), Artículo 2.